

Implementación de una Estrategia Integral de Pruebas para la Plataforma de Código Abierto alf.io de Venta de Entradas de Eventos

Christian Raúl Mestas Zagarra
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
cmestasz@unsa.edu.pe

Luis Gustavo Sequeiros Condori
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
lsequeiros@unsa.edu.pe

Mariel Alisson Jara Mamani
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
mjarama@unsa.edu.pe

Rodrigo Alexander Fernández
Huarca
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
rfernandezh@unsa.edu.pe

Álvaro Raúl Quispe Condori
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
aquispecondo@unsa.edu.pe

Mathías Alonso Barrios Medina
*Facultad de Ingeniería de la
Producción y Servicios*
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
mbarriosmed@unsa.edu.pe

Resumen—Este artículo presenta la implementación de una estrategia integral de pruebas para alf.io, una plataforma de código abierto para venta de entradas de eventos. El proyecto implicó diseñar y ejecutar pruebas en cuatro niveles: pruebas unitarias, de integración, de sistema y de aceptación. Las pruebas cubrieron toda la pila tecnológica incluyendo un backend Java/Spring Boot, frontend público Angular y una interfaz de administración basada en Lit. Las pruebas automatizadas a través de pipelines CI/CD de GitHub Actions alcanzaron tasas de aprobación del 100% en pruebas unitarias (2,290 casos), pruebas de integración (442 casos) y pruebas de extremo a extremo (88 casos), con una cobertura de código agregada del 85.76%. Las pruebas de rendimiento con K6 validaron el comportamiento del sistema bajo carga, el fuzzing con Jazzer confirmó la robustez del manejo de entradas, y el análisis estático con SonarQube identificó mejoras en la calidad del código. Los resultados demuestran la efectividad de los enfoques de pruebas multicapa para asegurar la calidad en aplicaciones web complejas.

Palabras clave—Pruebas de Software, Automatización de Pruebas, CI/CD, Código Abierto, Gestión de Eventos, Aseguramiento de Calidad

I. INTRODUCCIÓN

Las pruebas de software se han convertido en una práctica indispensable en la ingeniería de software moderna, garantizando que los sistemas cumplan con estándares de calidad y funcionen correctamente bajo diversas condiciones [1]. La complejidad de las aplicaciones web contemporáneas, que integran múltiples tecnologías, bases de datos y servicios externos, demanda estrategias de prueba integrales que trasciendan las pruebas unitarias básicas [2]. A medida que los sistemas crecen en complejidad, la necesidad de enfoques

sistemáticos de prueba que cubran todos los niveles se vuelve cada vez más crítica [3].

El dominio de gestión de eventos y venta de entradas presenta desafíos únicos para las pruebas de software. Sistemas como alf.io deben gestionar reservas concurrentes, procesar pagos a través de múltiples pasarelas, administrar la generación de entradas digitales y proporcionar funcionalidades de check-in en tiempo real [4]. Estos requisitos crean interacciones complejas entre componentes que solo pueden validarse mediante pruebas rigurosas en múltiples niveles. La naturaleza financiera de las transacciones de boletos añade otra capa de complejidad, dado que los errores pueden resultar en pérdidas de ingresos y daño a la reputación.

Este artículo presenta la implementación de una estrategia integral de pruebas para alf.io, una plataforma de código abierto para venta de entradas de eventos. El proyecto implicó diseñar y ejecutar pruebas en cuatro niveles distintos: pruebas unitarias, de integración, de sistema y de aceptación [5]. Cada nivel abordó atributos de calidad específicos y utilizó herramientas y metodologías especializadas para validar diferentes aspectos del comportamiento del sistema.

La implementación de pruebas cubrió toda la pila tecnológica de alf.io, incluyendo el backend Java/Spring Boot [6], el frontend público Angular y la interfaz de administración basada en Lit. La estrategia incorporó pruebas automatizadas a través de pipelines CI/CD de GitHub Actions [7], pruebas de rendimiento con K6 [8], fuzzing con Jazzer [9] y análisis estático con SonarQube [10]. Este enfoque multicapa aseguró cobertura integral de requisitos funcionales y no funcionales.

Los resultados demuestran la efectividad de la estrategia de pruebas implementada, alcanzando tasas de aprobación del 100% en pruebas unitarias, de integración y de extremo a extremo, con una cobertura de código agregada del 85.76% [11]. El proyecto también identificó y documentó defectos, validó contratos de API y estableció procesos de prueba reproducibles a través de pipelines automatizados [12]. Este trabajo proporciona una referencia práctica para implementar estrategias de pruebas integrales en aplicaciones web de código abierto similares [13].

II. ANTECEDENTES

A. Estado del Arte en Pruebas de Software

Las pruebas de software han evolucionado significativamente en las últimas décadas, transitando de la verificación manual a marcos y metodologías automatizadas sofisticadas [3]. El estándar ISO/IEC/IEEE 29119 proporciona un marco integral para las pruebas de software, definiendo conceptos, procesos, documentación y técnicas que guían las actividades de prueba en todos los niveles del desarrollo de software. Este estándar enfatiza la importancia de la planificación sistemática, ejecución y reporte de pruebas para asegurar la calidad del software.

El modelo de pirámide de pruebas, propuesto originalmente por Mike Cohn, permanece como un concepto fundamental en el diseño de estrategias de prueba [5]. Este modelo sugiere que los equipos deben invertir predominantemente en pruebas unitarias, seguido de pruebas de integración, con menos pruebas de extremo a extremo en la cima. Las adaptaciones modernas de este modelo consideran capas adicionales como pruebas de contrato y pruebas de rendimiento [14].

La automatización de pruebas se ha convertido en un pilar fundamental de las prácticas modernas de desarrollo de software, particularmente en el contexto de pipelines de integración continua y despliegue continuo [12]. La investigación ha demostrado que las pruebas automatizadas reducen significativamente el tiempo de comercialización mientras mejoran la calidad del código [15]. Los estudios han mostrado que los defectos encontrados en producción cuestan significativamente más de corregir que aquellos identificados durante el desarrollo.

Las estrategias modernas de pruebas abarcan múltiples enfoques especializados más allá de las pruebas funcionales tradicionales. Las herramientas de pruebas de rendimiento como K6 permiten a los equipos validar el comportamiento del sistema bajo diversas condiciones de carga [8]. Las pruebas de seguridad mediante herramientas de fuzzing como Jazzer pueden identificar vulnerabilidades que las pruebas tradicionales podrían pasar por alto [9]. Las herramientas de análisis estático como SonarQube proporcionan monitoreo continuo de calidad de código, detectando bugs, vulnerabilidades y code smells antes de llegar a producción [10].

B. Sistemas de Gestión de Eventos y Software de Código Abierto

Los sistemas de gestión de eventos representan una categoría especializada de aplicaciones web que deben manejar

lógica de negocio compleja, transacciones financieras y operaciones en tiempo real. El modelo de código abierto ha resultado particularmente exitoso en este dominio, ya que permite a las organizaciones personalizar y extender el software para satisfacer sus necesidades específicas mientras evitan el bloqueo de proveedores [16].

El proyecto alf.io, creado en 2014 por Celestino Bellone en Ticino, Suiza, ejemplifica la evolución del software de gestión de eventos de código abierto [4]. Lo que comenzó como una aplicación simple de Spring Boot ha crecido hasta convertirse en un ecosistema maduro con 1,596 estrellas en GitHub, más de 141,000 descargas en Docker Hub y 100 versiones. El nombre del proyecto deriva del nombre italiano Alfio, reflejando sus orígenes en la región de habla italiana de Suiza.

La pila tecnológica de alf.io ha evolucionado significativamente a lo largo de sus 12 años de historia. El backend transitó de Spring Boot 1.x a 3.x, con Java evolucionando de la versión 8 a la 17 [6]. El frontend experimentó una transformación importante de AngularJS a componentes web Lit, reflejando la tendencia general de la industria hacia frameworks ligeros y basados en estándares. La capa de base de datos se consolidó del soporte múltiple de bases de datos a PostgreSQL exclusivamente, aprovechando funciones avanzadas como Row-Level Security para multi-tenencia.

La arquitectura del proyecto sigue un diseño inspirado en microservicios manteniendo un modelo de despliegue monolítico. El backend implementa una arquitectura por capas con controladores, managers, repositorios y modelos. El frontend consiste en dos aplicaciones de página única separadas: una aplicación Angular pública para compras de entradas y una interfaz de administración basada en Lit para organizadores de eventos.

La integración de pasarelas de pago representa un aspecto crítico de los sistemas de gestión de eventos. Alf.io soporta múltiples proveedores de pago incluyendo Stripe, PayPal, Mollie y transferencias bancarias. El mecanismo de extensiones del sistema permite integraciones personalizadas y lógica de negocio, haciéndolo adaptable a diversos requisitos organizacionales.

III. CASO DE ESTUDIO

A. Historia y Evolución del Proyecto

Alf.io fue creado el 26 de julio de 2014 por Celestino Bellone, un desarrollador de software con base en Rovio, Suiza [4]. El proyecto surgió de la necesidad de una solución de venta de entradas autohospedada y de código abierto que pudiera personalizarse para diversos tipos de eventos. Bellone, quien posteriormente cofundó Swicket, una versión premium hospedada de alf.io, ha contribuido 2,531 commits al proyecto, representando aproximadamente el 64% de todas las contribuciones.

El co-mantenedor del proyecto, Sylvain Jermini, se unió en 2010 y ha contribuido 1,295 commits, representando el 33% del total de contribuciones. Jermini, con base en Suiza, ha sido instrumental en la evolución de la arquitectura del frontend, particularmente en la migración de AngularJS a componentes

web Lit. Juntos, Bellone y Jermini han mantenido un desarrollo continuo durante 12 años, lanzando 100 versiones y estableciendo alf.io como una solución madura y lista para producción [13].

La evolución del proyecto puede rastrearse a través de hitos importantes de versión. La Tabla I resume las versiones clave y sus características.

TABLA I
VERSIONES CLAVE Y HITOS DE ALF.IO.

Versión	Fecha	Características Clave
1.0	2014	Versión inicial
1.14	Abr 2018	Sistema de extensiones con hooks de scripting
2.0-M0	Nov 2018	Spring Boot 2, eliminación de MySQL/HSQLDB
2.0-M4	Abr 2022	Organizaciones, multi-tenencia, Stripe/Mollie/PayPal
2.0-M5	Sep 2024	Spring Boot 3.2, frontend admin Lit, Cloudflare Turnstile
2.0-M5-2606	Jun 2026	Correcciones de seguridad, pagos offline personalizados, frontend pnpm

B. Pila Tecnológica

La pila tecnológica actual de alf.io refleja las prácticas modernas de desarrollo de aplicaciones web [6]. La Tabla II resume los componentes principales.

TABLA II
PILA TECNOLÓGICA DE ALF.IO.

Componente	Tecnología
Backend	Java 17, Spring Boot 3.5.x, Jetty
Sistema de Build	Gradle con integración de automatización de pruebas
Base de Datos	PostgreSQL 10/15/16 con migraciones Flyway
Frontend Público	Angular 17
Frontend Admin	Lit 3 con Shoenice, Vite
Pruebas Unitarias	JUnit 5, Vitest, Mockito
Pruebas de Integración	Testcontainers [17], Playwright [18]
Pruebas de Rendimiento	K6 [8]
Fuzzing	Jazzer [9]
Análisis Estático	SonarQube [10]

C. Arquitectura del Sistema

La arquitectura del sistema sigue un patrón de diseño por capas con separación clara de responsabilidades. El backend implementa una arquitectura estándar de Spring Boot con controladores que manejan solicitudes HTTP, managers que implementan lógica de negocio, repositorios que gestionan interacciones con la base de datos y modelos que representan entidades de dominio [6].

El esquema de la base de datos implementa Row-Level Security en PostgreSQL para asegurar el aislamiento de

datos entre organizaciones en despliegues multi-tenancy. Este enfoque proporciona seguridad a nivel de base de datos sin requerir instancias separadas de base de datos para cada tenant. El esquema está controlado por versiones a través de migraciones Flyway, asegurando un estado consistente de la base de datos entre ambientes de desarrollo, pruebas y producción.

Las integraciones con servicios externos incluyen pasarelas de pago, servicios de correo electrónico y proveedores de billeteras digitales. El sistema soporta tanto métodos de pago en línea como fuera de línea, con mecanismos incorporados para manejar confirmación de pagos, expiración y reconciliación.

D. Fork y Contexto Académico

El repositorio en estudio, catarinas-ps-2026/alf.io, es un fork del proyecto original alfio-event/alf.io, creado con fines académicos. El fork fue establecido el 24 de mayo de 2026, como parte de un curso de pruebas de software en la Universidad Nacional de San Agustín de Arequipa. El equipo del proyecto consta de seis estudiantes liderados por un líder de pruebas, con el profesor Robert Edison Arisaca Mamani proporcionando supervisión académica.

El contexto académico requirió un enfoque sistemático de pruebas que trascendiera las prácticas típicas de contribución a código abierto [16]. El equipo implementó planes de pruebas integrales siguiendo los estándares ISO/IEC/IEEE 29119, con documentación formal de estrategias de prueba, resultados de ejecución y seguimiento de defectos [3].

IV. METODOLOGÍA

La estrategia de pruebas siguió un enfoque estructurado con siete actividades principales, cada una abordando un aspecto específico del proceso de aseguramiento de calidad. Esta sección describe cada actividad en detalle.

A. Planificación y Metodología

La planificación de pruebas siguió el estándar ISO/IEC/IEEE 29119, con planes de pruebas formales que documentaban alcance, metodología, criterios de entrada y salida, y estrategias de mitigación de riesgos [3]. El proyecto adoptó Scrum como marco de desarrollo, con sprints de dos semanas y entregables claramente definidos para cada nivel de prueba [5]. La comunicación se mantuvo a través de WhatsApp para coordinación diaria, Google Meet para ceremonias de sprint y GitHub Projects para seguimiento de tareas.

Cada nivel de prueba tuvo su propio plan de pruebas e informe de ejecución, proporcionando trazabilidad desde los requisitos hasta los resultados de las pruebas. El equipo realizó reuniones de planificación, revisión y retrospectiva de sprint para asegurar alineamiento y mejora continua.

La gestión de riesgos se integró en el proceso de pruebas, con un registro formal de riesgos que identificaba problemas potenciales y sus estrategias de mitigación. La Tabla III resume los riesgos clave identificados.

TABLA III
RIESGOS CLAVE IDENTIFICADOS Y SUS ESTRATEGIAS DE MITIGACIÓN.

Riesgo	Prob.	Impacto	Severidad	Mitigación
Experiencia limitada en Spring Boot	4	4	16	Revisiones de código por pares
Fecha de entrega ajustada	5	4	20	Priorizar módulos críticos
Conflictos de integración	4	3	12	Políticas estrictas de branching
Dificultades con automatización	2	3	6	Soporte técnico interno

B. Pruebas Unitarias

Las pruebas unitarias se enfocaron en validar componentes individuales de forma aislada, utilizando mocks y dobles de prueba para eliminar dependencias externas [2]. Las pruebas del backend utilizaron JUnit 5 con Mockito para mocking de dependencias, mientras que las pruebas del frontend utilizaron Vitest con HttpTestingController para Angular y jsdom para componentes Lit [6].

La suite de pruebas se organizó por capa de aplicación: managers, controladores, repositorios, utilidades y componentes del frontend. Las pruebas unitarias del backend cubrieron managers de lógica de negocio, controladores API REST, repositorios de datos y clases de utilidad. Las pruebas unitarias del frontend validaron comportamiento de componentes, comunicación de servicios, guards de rutas y validaciones de formularios.

La ejecución de pruebas se automatizó a través de GitHub Actions [7], con dos workflows: test-pr.yml para pull requests y test-push.yml para pushes a la rama main. La cobertura se midió usando JaCoCo para backend y coverage-v8 de Vitest para frontend [11].

C. Pruebas de Integración

Las pruebas de integración validaron la interacción entre múltiples componentes usando dependencias reales [17]. La estrategia adoptó un enfoque Big Bang incremental, integrando primero componentes de infraestructura, luego flujos de lógica de negocio y finalmente validación de contratos de API.

Las pruebas de integración del backend utilizaron Testcontainers para gestionar instancias efímeras de PostgreSQL, proporcionando interacciones reales con la base de datos sin dependencias externas. Las pruebas validaron flujos completos desde controladores hasta repositorios a través de managers. El contenedor Stripe Mock se utilizó para pruebas de integración con pasarelas de pago, mientras que cuentas sandbox de PayPal permitieron la validación de flujos de pago reales.

La validación de contratos de API se realizó usando SpringDoc OpenAPI para generar descriptores automáticamente desde los controladores. Las pruebas de extremo a extremo con Playwright validaron la interfaz de administración contra el backend en ejecución [18].

D. Pruebas Funcionales

Las pruebas funcionales emplearon técnicas de caja negra para validar el comportamiento del sistema contra requisitos sin examinar la estructura interna del código [2]. Las técnicas aplicadas incluyeron partición por equivalencia, análisis de valores límite, pruebas de casos de uso y tablas de decisión [3].

El proyecto fue encargado de seleccionar tres módulos funcionales para pruebas enfocadas. Los módulos seleccionados fueron: Manager, Controller/API y Flujos de Reserva. Estos módulos representan la lógica de negocio principal, la capa API y los flujos críticos de usuario del sistema respectivamente.

La suite de pruebas funcionales cubrió casos de prueba manuales en los tres módulos seleccionados, con cobertura adicional de módulos de soporte. Cada caso de prueba siguió un formato estructurado con precondiciones, datos de entrada, pasos de ejecución y resultados esperados.

E. Pruebas de Sistema

Las pruebas de sistema se enfocaron en requisitos no funcionales, específicamente seguridad y rendimiento [1]. La estrategia de pruebas de seguridad empleó dos enfoques complementarios: pruebas de seguridad de aplicación estática con SonarQube [10] y fuzzing con Jazzer [9]. Las pruebas de rendimiento utilizaron K6 para validar el comportamiento del sistema bajo diversas condiciones de carga [8].

La Tabla IV resume las herramientas utilizadas para las pruebas de sistema.

TABLA IV
HERRAMIENTAS DE PRUEBAS DE SISTEMA Y SUS OBJETIVOS.

Herramienta	Categoría	Objetivo	Métrica
SonarQube	SAST - Seguridad	Análisis estático de código	Bugs, vulnerabilidades, code smells
Jazzer	Fuzzing - Seguridad	35 clases de utilidad	Crashes, excepciones
K6	Rendimiento	Flujo de reserva	Tiempo de respuesta, throughput, tasa de error

F. Pruebas de Aceptación

Las pruebas de aceptación validaron flujos de trabajo completos desde la perspectiva del usuario final. La suite de pruebas de aceptación cubrió la interfaz del panel de administración, incluyendo autenticación, gestión de eventos, procesamiento de reservas y operaciones de check-in. Estas pruebas se ejecutaron contra una instancia de alf.io desplegada en un clúster de Kubernetes.

Las pruebas de aceptación utilizaron Playwright para automatización de navegadores, probando contra navegadores Chromium y Firefox [18]. Las pruebas validaron que el sistema cumplía con los criterios de aceptación especificados y funcionaba correctamente en un ambiente similar al de producción.

Test Summary



Ignored tests	Packages	Classes			
Class	Tests	Failures	Ignored	Duration	Success rate
aflo.DashboardManagementTest	2	0	0	0.181s	100%
aflo.DashboardManagementTest	2	0	0	0.088s	100%
aflo.controller.api.CheckTestDashboardIntegrationTest	1	0	0	8.700s	100%
aflo.controller.api.admin.AdminReservationApiControlIntegrationTest	6	0	0	4.530s	100%
aflo.controller.api.admin.CheckApiControlIntegrationTest	17	0	0	8.210s	100%
aflo.controller.api.admin.ControllerApiControlIntegrationTest	14	0	0	4.350s	100%
aflo.controller.api.admin.EventApiControlIntegrationTest	10	0	0	3.244s	100%
aflo.controller.api.admin.EventApiControlIntegrationTest	2	0	0	0.725s	100%
aflo.controller.api.v1.ConfigurationApiV1IntegrationTest	3	0	0	0.682s	100%
aflo.controller.api.v1.EventApiV1IntegrationTest	8	0	0	2.154s	100%
aflo.controller.api.v1.ReservationApiV1ControlIntegrationTest	14	0	0	6.246s	100%
aflo.controller.api.v1.SubscriptionApiV1IntegrationTest	3	0	0	0.546s	100%
aflo.controller.api.v2.LoginEventApiV2ControlIntegrationTest	9	0	0	1.994s	100%
aflo.controller.api.v2.LoginEventApiV2ControlIntegrationTest	7	0	0	2.459s	100%
aflo.controller.api.v2.LoginReservation.BillingDocumentCreationIntegrationTest	7	0	0	3.890s	100%
aflo.controller.api.v2.LoginReservation.CustomOrderReservationIntegrationTest	2	0	0	4.299s	100%
aflo.controller.api.v2.LoginReservation.CustomOrderReservationIntegrationTest	2	0	0	0.761s	100%
aflo.controller.api.v2.LoginReservation.DiscountReservationIntegrationTest	1	0	0	2.955s	100%
aflo.controller.api.v2.LoginReservation.HybridEventReservationIntegrationTest	1	0	0	2.698s	100%

Fig. 5. Resultados de pruebas de integración del backend (JUnit).

La suite de pruebas de integración validó 30 endpoints obligatorios cubriendo el flujo crítico: Reserva, Pago y Check-In. La Figura 6 muestra los resultados de las pruebas Playwright [18].

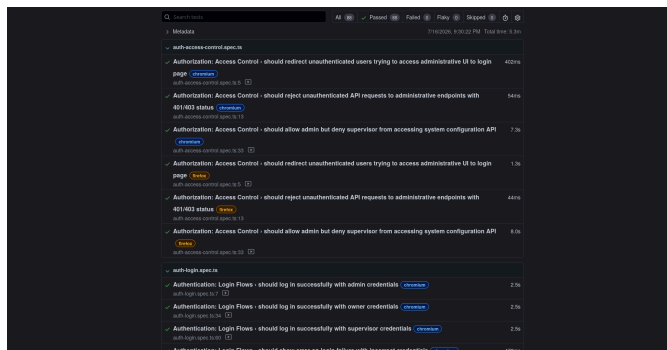


Fig. 6. Resultados de pruebas Playwright (Chromium + Firefox).

La validación de contratos de API confirmó la estabilidad entre la implementación actual y el descriptor de referencia. La Figura 7 muestra el portal de validación de contratos de API.

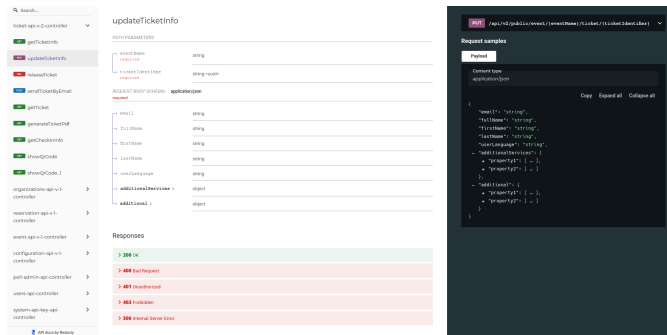


Fig. 7. Portal de validación de contratos API (Redoc/OpenAPI).

D. Resultados de Pruebas Funcionales

Las pruebas funcionales cubrieron casos de prueba manuales en los tres módulos seleccionados: Manager, Controller/API y Flujos de Reserva [2]. La tasa de aprobación global de pruebas funcionales fue del 97.5%. La Tabla VI resume los resultados por módulo.

TABLA VI
RESULTADOS DE PRUEBAS FUNCIONALES POR MÓDULO.

Módulo	Casos de Prueba	Tasa de Aprobación	Defectos Encontrados	En-
Manager	15	93%	1	
Controller/API	20	95%	1	
Flujos de Reserva	25	96%	1	
Edición de Tickets	11	82%	3	
Configuración de Eventos	7	57%	3	
Otros Módulos	42	100%	0	

E. Defectos Encontrados y Corregidos

El proceso de pruebas identificó y documentó defectos en todos los niveles de prueba [3]. La Tabla VII lista los defectos descubiertos durante las pruebas funcionales y los commits que los corrigieron.

TABLA VII
DEFECTOS DESCUBIERTOS EN PRUEBAS FUNCIONALES Y SUS CORRECCIONES.

ID de Prueba	Módulo	Defecto	Commit	Estado
CPF-01-003	Edición de Tickets	Error 500 con nombres de 254 caracteres	7e524f8	Corregido
CPF-01-006	Edición de Tickets	Sistema acepta números en campo nombre	99f17a7	Corregido
CPF-01-011	Edición de Tickets	Validación de longitud de email no aplicada	dc1341a	Corregido
CPF-03-006	Estado de Reserva	Botón visible con datos incompletos	7e524f8	Corregido
CPF-09-006	Config. Organización	Error SQL expuesto con nombres de 256 caracteres	99f17a7	Corregido
CPF-10-001	Config. Evento	Bug de scroll en selector de fechas	dc1341a	Corregido
CPF-10-006	Config. Evento	Guardado silencioso deshabilitado sin retroalimentación	7e524f8	Corregido
CPF-10-007	Config. Evento	Códigos ocultos duplicados permitidos	99f17a7	Corregido
CPF-14-004	Localización	Advertencia de zona horaria no mostrada	dc1341a	Corregido

F. Resultados de Pruebas de Sistema

La Figura 8 muestra los resultados de pruebas de rendimiento K6 [8]. Las pruebas de rendimiento ejecutaron 54,176 peticiones HTTP contra el flujo de reserva, alcanzando un throughput de 190.43 peticiones por segundo. El tiempo de respuesta mediano fue de 745 milisegundos, cumpliendo con el objetivo de menos de 2 segundos. Sin embargo, la tasa de error del 8.43% excedió el objetivo de menos del 1% bajo condiciones de carga extrema.

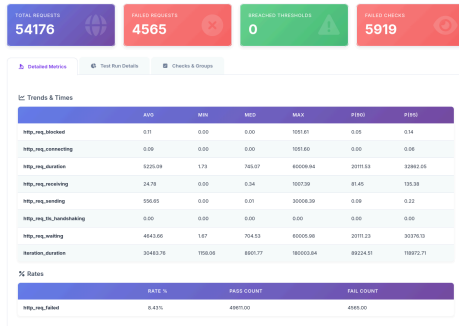


Fig. 8. Resultados de pruebas de rendimiento K6.

La Figura 9 muestra los resultados de fuzzing con Jazzer [9]. Las pruebas de fuzzing ejecutaron 35 clases objetivo cubriendo deserialización JSON, validación de entrada, utilidades criptográficas, renderizado de plantillas, lógica de negocio y procesamiento de extensiones [19]. La campaña de fuzzing encontró cero crashes críticos.

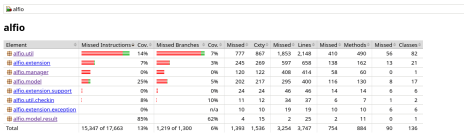


Fig. 9. Resultados de pruebas de fuzzing Jazzer.

La Figura 10 muestra los resultados de análisis estático SonarQube [10]. El análisis estático identificó 16 bugs, 10 vulnerabilidades y 1,412 code smells en el código fuente. El Quality Gate pasó con un ratio de deuda técnica del 0.5% [20].

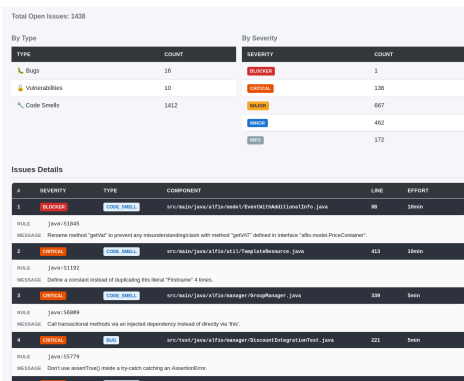


Fig. 10. Resultados de análisis estático SonarQube.

VI. CONCLUSIÓN

La estrategia integral de pruebas implementada para alf.io demostró la efectividad de un enfoque multicapa para el aseguramiento de calidad de software. El proyecto alcanzó tasas de aprobación del 100% en pruebas unitarias, de integración y de extremo a extremo, manteniendo una cobertura de código agregada del 85.76%. Estos resultados validaron la metodología de pruebas y confirmaron la corrección funcional del sistema.

La fase de pruebas unitarias estableció una base sólida validando componentes individuales de forma aislada. La combinación de JUnit 5 con Mockito para backend y Vitest para frontend proporcionó cobertura integral de lógica de negocio, controladores API y componentes de interfaz de usuario. La ejecución automatizada a través de GitHub Actions aseguró resultados consistentes de pruebas y retroalimentación rápida para los desarrolladores.

Las pruebas de integración confirmaron que los componentes del sistema interactuaban correctamente con dependencias reales. El uso de Testcontainers para instancias efímeras de PostgreSQL, Stripe Mock para pruebas de pago y Playwright para automatización de navegadores proporcionó validación realista del stack completo del sistema. La validación de contratos de API a través de OpenAPI y Redoc aseguró la estabilidad de la API entre versiones.

La fase de pruebas funcionales, conducida mediante ejecución manual de más de 120 casos de prueba, validó la funcionalidad orientada al usuario en 15 módulos. Estas pruebas identificaron defectos que las pruebas automatizadas no pudieron detectar, como inconsistencias en el comportamiento de la interfaz de usuario y casos extremos en interacciones de usuario. La aplicación sistemática de técnicas de caja negra aseguró cobertura integral de requisitos funcionales.

Las pruebas de sistema abordaron requisitos no funcionales mediante pruebas de rendimiento con K6, fuzzing con Jazzer y análisis estático con SonarQube. Si bien las pruebas de rendimiento identificaron un problema de escalabilidad bajo carga extrema, el sistema demostró degradación elegante. El fuzzing confirmó el manejo robusto de entradas inválidas con cero crashes críticos, y SonarQube identificó áreas de mejora en calidad de código con un ratio de deuda técnica aceptable del 0.5%.

La implementación estableció procesos de prueba reproducibles mediante pipelines CI/CD automatizados, documentación integral siguiendo estándares ISO/IEC/IEEE 29119 y seguimiento sistemático de defectos. El proyecto demostró que las estrategias de pruebas integrales, cuando se planifican y ejecutan adecuadamente, pueden mejorar significativamente la calidad del software mientras proporcionan retroalimentación valiosa para equipos de desarrollo trabajando en aplicaciones web complejas.

VII. TRABAJO FUTURO

La estrategia de pruebas implementada para alf.io proporciona una base para la mejora continua y expansión. Se han

identificado varias áreas para mejora futura que fortalecerían aún más el proceso de aseguramiento de calidad.

Las pruebas de rendimiento deberían expandirse para abordar los problemas de escalabilidad identificados bajo condiciones de carga extrema [8]. Las pruebas K6 actuales revelaron una tasa de error que excedió el objetivo con 1,600 usuarios concurrentes. El trabajo futuro debería investigar las causas raíz en los endpoints de reserva y confirmación, implementar mecanismos de rate limiting y optimizar consultas de base de datos para mejorar el throughput.

La fase de pruebas de aceptación debería expandirse para incluir validación completa de extremo a extremo de flujos de trabajo de usuario [18]. Esto incluye pruebas de aceptación automatizadas que simulen recorridos reales de usuario desde el descubrimiento de eventos hasta la compra de entradas, check-in y reportes posteriores al evento. Tales pruebas proporcionarían mayor confianza en la preparación del sistema para producción.

Las pruebas de seguridad podrían mejorarse expandiendo la cobertura de fuzzing para incluir componentes adicionales del sistema [9] e implementando pruebas de penetración enfocadas en seguridad. El análisis actual de SonarQube identificó vulnerabilidades que deberían abordarse, particularmente el problema BLOCKER relacionado con conflictos de nombres de métodos y los problemas CRITICAL que involucran llamadas transaccionales [10].

La cobertura de automatización de pruebas debería expandirse para incluir más escenarios funcionales que actualmente se ejecutan manualmente [2]. Convertir casos de prueba manuales en scripts automatizados mejoraría la eficiencia de pruebas de regresión y permitiría ciclos de liberación más rápidos [12]. La prioridad debería darse a casos de prueba de alto impacto que cubran procesamiento de pagos, generación de entradas y operaciones de check-in.

La infraestructura de pruebas podría beneficiarse de la implementación de pruebas de regresión visual para detectar cambios no intencionados en la interfaz de usuario. Herramientas como Percy o Chromatic podrían integrarse en la suite de pruebas de Playwright para capturar y comparar capturas de pantalla entre versiones de navegadores y tamaños de pantalla.

El monitoreo y observabilidad deberían integrarse en el proceso de pruebas para proporcionar información en tiempo real sobre el comportamiento del sistema durante la ejecución de pruebas. Implementar tracing distribuido, recolección de métricas y agregación de logs permitiría un mejor diagnóstico de fallos en pruebas y problemas de rendimiento.

La documentación generada durante este proyecto, siguiendo estándares ISO/IEC/IEEE 29119, proporciona una plantilla para esfuerzos futuros de pruebas [3]. Esta documentación debería mantenerse y actualizarse a medida que el sistema evolucione, asegurando que los planes de pruebas, informes de ejecución y seguimiento de defectos permanezcan actuales y útiles para actividades continuas de aseguramiento de calidad.

REFERENCIAS

- [1] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill Education, 2014.
- [2] G. J. Myers, T. Badgett, and T. M. Thomas, *The Art of Software Testing*, 3rd ed. John Wiley & Sons, 2011.
- [3] ISO/IEC/IEEE, *ISO/IEC/IEEE 29119:2021 - Software Testing*. International Organization for Standardization, 2021.
- [4] C. Bellone, "alf.io - Free and Open Source Event Attendance Management," 2014.
- [5] M. Cohn, *Agile Estimating and Planning*. Upper Saddle River, NJ: Prentice Hall, 2004.
- [6] VMware, "Spring Boot Documentation," 2024.
- [7] GitHub, "GitHub Actions Documentation," 2024.
- [8] I. W. R. Pinastawa and others, "Web Server Performance Evaluation: Comparing Nginx and Apache Using K6 Testing Methods for Load, Spike, Soak, and Performance," in *2024 IEEE International Conference on Innovation, Analytics, Intelligent Technologies and Systems (ICI-CiTIS)*, 2024. doi: 10.1109/icimcis63449.2024.10956402.
- [9] S. Dechand and others, "Jazzer: Coverage-Guided Fuzzing for Semantic Vulnerabilities in the Java Ecosystem," in *IEEE Symposium on Security and Privacy (S&P)*, 2026. doi: 10.1109/SP63933.2026.00134.
- [10] D. Marclio and others, "Are Static Analysis Violations Really Fixed? A Closer Look at Realistic Usage of SonarQube," in *2019 IEEE/ACM International Conference on Program Comprehension (ICPC)*, 2019. doi: 10.1109/icpc.2019.00040.
- [11] P. S. Kochhar and others, "Code Coverage and Test Suite Effectiveness: Empirical Study with Real Bugs in Large Systems," in *2015 IEEE 22nd International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2015. doi: 10.1109/saner.2015.7081877.
- [12] S. Elbaum and others, "Techniques for Improving Regression Testing in Continuous Integration Development Environments," in *Proc. 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2016. doi: 10.1145/2635868.2635910.
- [13] M. Aberdour, "Achieving Quality in Open-Source Software," *IEEE Software*, 2007, doi: 10.1109/ms.2007.2.
- [14] V. Mukhin and others, "The Testing Mechanism for Software and Services Based on Mike Cohn's Testing Pyramid Modification," in *2021 IEEE 11th International Conference on Intelligent Data Acquisition and Advanced Computing Systems (IDAACS)*, 2021. doi: 10.1109/idaacs53288.2021.9660999.
- [15] H. Spieker and others, "Reinforcement Learning for Automatic Test Case Prioritization and Selection in Continuous Integration," in *Proc. 2017 10th International Conference on Software Testing, Verification and Validation (ICST)*, 2017. doi: 10.1145/3092703.3092709.
- [16] I. Stamelos and others, "Code Quality Analysis in Open Source Software Development," *Information Systems Journal*, 2002, doi: 10.1046/j.1365-2575.2002.00117.x.
- [17] S. Shahsavan and others, "Testing the Real World: Integration Tests with Testcontainers in Go," 2026, doi: 10.1007/979-8-8688-2370-1_17.
- [18] G. Aryotejo and others, "A Comparative Analysis of Web Automation Testing Tools: Selenium Webdriver, Playwright, and Webdriverio Using an AHP-TOPSIS Approach," in *IEEE International Conference (ICICoS)*, 2025. doi: 10.1109/icicos68590.2025.11330144.
- [19] A. Takanen, J. D. DeMott, and C. Miller, *Fuzzing for Software Security Testing and Quality Assurance*. Artech House, 2008.
- [20] V. Lenarduzzi and others, "Are SonarQube Rules Inducing Bugs?," 2020, doi: 10.1109/saner48275.2020.9054821.